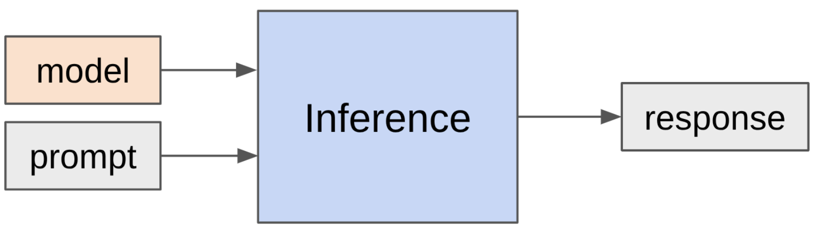
```python
from dataclasses import dataclass

from sympy import symbols, oo
from edtrace import text, link, image
from lecture_util import article_link
from references import Reference, gqa_2023, mla_2024, longformer_2020, sparse_transformer_2019, mistral_7b_2023, deepseek_v4_2026

# Define symbols corresponding to the shape of the Transformer model
B, S, T, D, F, N, K, H, L, V = symbols("B S T D F N K H L V", positive=True)
c = symbols("c", positive=True)  # Just a constant that helps with taking limits
memory_bandwidth = symbols("memory_bandwidth", positive=True)

scaling_book_transformers = Reference(title="Scaling book chapter on Transformers", url="https://jax-ml.github.io/scaling-book/transformers/")
scaling_book_inference = Reference(title="Scaling book chapter on inference", url="https://jax-ml.github.io/scaling-book/inference/")

def main():
```

Lecture 10: inference



Understanding the inference workload

```python
landscape()
review_transformer()
review_of_arithmetic_intensity()
arithmetic_intensity_of_inference()
throughput_and_latency()
```

Taking shortcuts (lossy)

```python
reduce_kv_cache_size()
quantization()
model_pruning()
```

Summary: reduce inference complexity without hurting accuracy

From scratch recipe:
1. Define faster model architecture
2. Train faster model

Distillation recipe:
1. Define faster model architecture

2. Initialize weights using original model (which has a different architecture)
3. Repair faster model (distillation)

Use shortcuts but double check (lossless)

```
speculative_sampling()
```

Handling dynamic workloads

Batching over sequences in live traffic is tricky because:
1. Requests arrive at different times (waiting for batch is bad for early requests)
2. Sequences have shared prefixes (e.g., system prompts, generating multiple samples)
3. Sequences have different lengths (padding is inefficient)

```
continuous_batching()
paged_attention()
```

Summary

- Inference is important (actual use, evaluation, reinforcement learning)
- Different characteristics compared to training (memory-bound, dynamic)
- Techniques: new architectures, quantization, pruning/distillation, speculative sampling
- Ideas from systems (speculative execution, paging)
- New architectures have huge potential for improvement

```
def landscape():
```

Inference shows up in many places:
- Actual use (chatbots, code completion, agents, batch data processing)
- Model evaluation (e.g., on instruction following)
- Reinforcement learning (sample many generations, then apply score)

Why **efficiency** matters: training is one-time cost, inference is repeated many times
- OpenAI processes ~8.6T tokens per day [article]
- For reference, DeepSeek v4 was trained on 32T tokens
[DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence]

Moreover:
- Chatbots: most tokens are meant for human consumption (humans are bottleneck)
- Agents: query → internal trace → output for human (number of tokens generated can grow unbounded)
- Tokens generated = compute spent

Companies doing inference (a big deal for anyone who has a product or platform):
- Providers serving closed models (OpenAI, Anthropic, Google, etc.)
- Providers serving open-weight models (Together, Fireworks, Baseten, DeepInfra, Groq, Cerebras, etc.)

Open-source packages:
- vLLM: from Berkeley, pioneered PagedAttention, popular and good default [GitHub]
- SGLang: from Berkeley, pioneered RadixAttention, good for agentic workloads [project]
- TensorRT-LLM: from NVIDIA, highly optimized for GPUs [article]
- llama.cpp: C++ only, supports CPU inference, runs locally [GitHub]

Inference is huge. Important to make it fast.

What does "fast" mean (metrics)?

- Time-to-first-token (TTFT): how long user waits before any generation happens (for interactive applications)
- Latency (seconds/token): how fast tokens appear for *one* query (for interactive applications)
- Throughput (tokens/second): how fast tokens appear for *many* queries (for batch processing)

What governs efficiency?

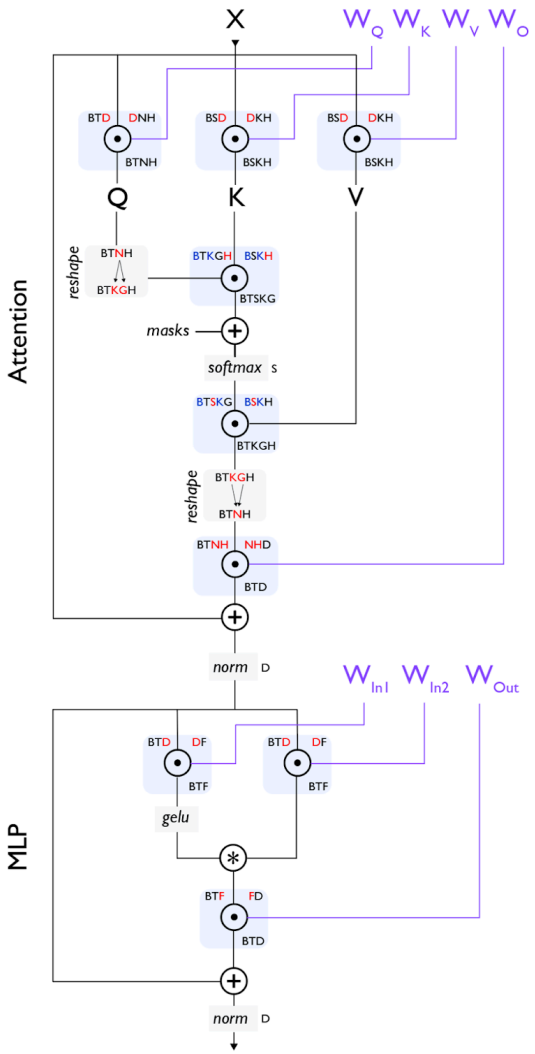
- Training (supervised): you see all tokens, can parallelize over sequence (matmul in Transformer)
- Inference: you have to generate sequentially, can't parallelize over generation, so harder to fully utilize compute

```
def review_transformer():
```

[Scaling book chapter on Transformers]

Notation (similar to einops):

- Symbols denote dimensions (and their length): B (batch), T (sequence), D (model dim), H (head dim)
- Example: BTD x DH → BTH
- Contracting (red) dimensions appear in both operands and disappear from result
- Regular (black) dimensions appear in one operand and stay in result
- Example: BD x BD → B
- Batching (blue) dimensions appear in both operands and stay in result

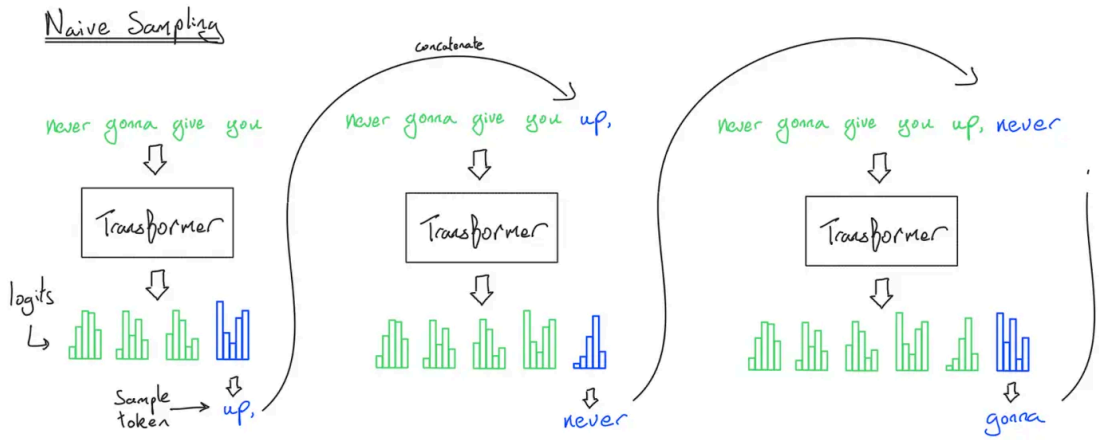


symbol	dimension
B	batch
L	number of layers
T	sequence length (query)
S	sequence length (key value)
V	vocab
D	d_model, embedding dimension
F	MLP hidden dimension
H	attention head dimension
N	number of query heads
K	number of key/value heads
G	q heads per kv head = N // K

Conventions:

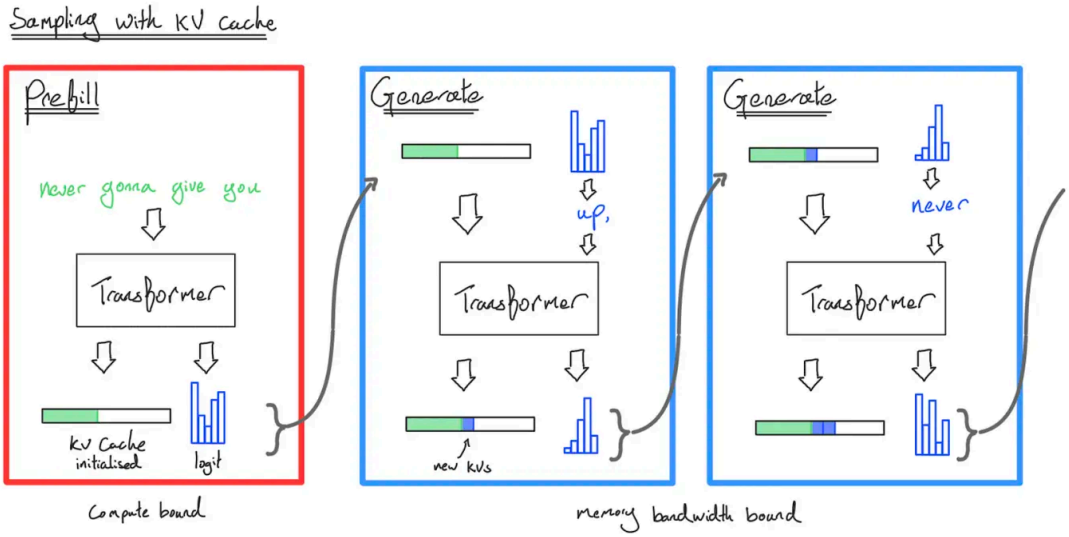
- F = 4 D (MLP up-projects into 4x the model dimension)
- D = N H (model dimension split across N heads)
- N = K G (for GQA, number of heads split across K groups)
- S = T (during training, condition on S input tokens to predict T output tokens)

```
def review_of_arithmetic_intensity():
    Setup: multiply X (B x D) and W (D x F) matrix
    Intuition: B is batch size, D is hidden dimension, F is up-projection dimension in MLP

    Let's do FLOPs and memory read/write accounting for the matrix multiplication (X * W).
    flops = 0
    bytes_transferred = 0

    # Perform the operation
    1. Read X (B x D) from HBM
    bytes_transferred += 2*B*D   # 2 bytes for bf16
    2. Read W (D x F) from HBM
    bytes_transferred += 2*D*F
    3. Compute Y = X (B x D) @ W (D x F)
    flops += 2*B*D*F
    4. Write Y (B x F) to HBM
    bytes_transferred += 2*B*F

    assert flops == 2*B*D*F
    assert bytes_transferred == 2*B*D + 2*D*F + 2*B*F

    Recall that arithmetic intensity is how much compute we do per byte transferred (want to be high).
    intensity = (flops / bytes_transferred).simplify()

    Assuming B is much less than D and F, then we can simplify:
    intensity = intensity.subs(D, c*B).subs(F, c*B).limit(c, oo).simplify()
    assert intensity == B

    Accelerator intensity of H100:
    flops_per_second = 989e12
    memory_bandwidth = 3.35e12
    accelerator_intensity = flops_per_second / memory_bandwidth
    assert round(accelerator_intensity) == 295

    If computation intensity > accelerator intensity, compute-bound (good)
    If computation intensity < accelerator intensity, memory-bound (bad)
    Conclusion: compute-bound iff B > 295

    Extreme case (B = 1, corresponding to matrix-vector product):
    - Arithmetic intensity: 1
    - Memory-bound (read D x F matrix, perform only 2 D F FLOPs)
    - This is basically what happens with inference...


def arithmetic_intensity_of_inference():
    [Scaling book chapter on inference]
```



Naive inference: to generate each token, feed history into Transformer
Complexity: generating T tokens requires O(T^3) FLOPs (one feedforward pass is O(T^2))

Observation: a lot of the work can be shared across prefixes
Solution: store **KV cache** in HBM



KV cache: for every sequence (B), token (S), layer (L), head (K), store an H-dimensional vector

Two stages of inference:
1. **Prefill**: given a prompt, encode into vectors (parallelizable like in training)
2. **Generation**: generate new response tokens (sequential)

Let's compute the FLOPs and memory IO for both the MLP and attention layers.
S is the number of tokens we're conditioning on, T is the number of tokens we're generating.
Later, we'll specialize to prefill (T = S) and generation (T = 1).

MLP layers (only looking at the matrix multiplications)

```
flops = 0
bytes_transferred = 0

# Perform the operation
```

1. Read X (B x T x D) from HBM

```
bytes_transferred += 2*B*T*D
```

2. Read Wup (D x F), Wgate (D x F), Wdown (F x D) from HBM

```
bytes_transferred += 3 * 2*D*F
```

3. Compute U = X (B x T x D) @ Wup (D x F)

```
flops += 2*B*T*D*F
```

4. Write U (B x T x F) to HBM

```
bytes_transferred += 2*B*T*F
```

5. Compute G = X (B x T x D) @ Wgate (D x F)

```
flops += 2*B*T*D*F
```

6. Write G (B x T x F) to HBM

```
bytes_transferred += 2*B*T*F
```

7. Compute Y = GeLU(G)*U (B x T x F) @ Wdown (F x D)

```
flops += 2*B*T*D*F
```

8. Write Y (B x T x D) to HBM

```
bytes_transferred += 2*B*T*D

assert flops == 6*B*T*D*F
assert bytes_transferred == 4*B*T*D + 4*B*T*F + 6*D*F

# Compute the arithmetic intensity
intensity = (flops / bytes_transferred).simplify()
```

Assume that B*T is much smaller than D and F.

```
intensity = intensity.subs(D, c*B*T).subs(F, c*B*T).limit(c, oo).simplify()
assert intensity == B*T
```

For the two stages:

1. Prefill: easy to make compute-bound (good) by making `B*T` large enough (large batches, long sequences)
2. Generation: two problems

- Generating one token at a time (T = 1)
- B is number of concurrent requests, unpredictable for interactive applications

Attention layers (focusing on the matrix multiplications with FlashAttention)

- S is number of previous tokens (already generated)
- T is number of next tokens (to generate logits for)

```
flops = 0
bytes_transferred = 0

# Perform the operation
```

1. Read Q (B x T x D), K (B x S x D), V (B x S x D) from HBM

```
bytes_transferred += 2*B*T*D + 2*B*S*D + 2*B*S*D
```

2. Compute A = Q (B x T x D) @ K (B x S x D)

```
flops += 2*B*S*T*D
```

3. Compute Y = softmax(A) (B x S x T x K x G) @ V (B x S x K x H)

```
flops += 2*B*S*T*D
```

4. Write Y (B x T x D) to HBM

```
bytes_transferred += 2*B*T*D

assert flops == 4*B*S*T*D
assert bytes_transferred == 4*B*S*D + 4*B*T*D

# Compute the arithmetic intensity
intensity = (flops / bytes_transferred).simplify()
assert intensity == S*T / (S + T)
```

For the two stages:

1. Prefill: T = S

```
prefill_intensity = intensity.subs(T, S).simplify()
assert prefill_intensity == S/2  # Good!
```

2. Generation: T = 1

```
generate_intensity = intensity.subs(T, 1).simplify()
```

```
    assert generate_intensity < 1  # Bad!
```

Unlike MLPs, no dependence on B, so batching doesn't help!
Why?

- In MLP layers, every sequence hits the same MLP weights (Wup, Wgate, Wdown don't depend on B)
- In attention layers, every sequence has its own KV cache vectors (Q, K, V all depend on B)

Summary:

- Prefill is compute-bound, generation is memory-bound
- Prefill MLP intensity: `B*S`
- Prefill attention intensity: `S/2`
- Generation MLP intensity: `B` (requires concurrent requests)
- Generation attention intensity: `<1` (impossible to improve)

```
@dataclass(frozen=True)
class TransformerPerformanceStats:
    """
    Performance stats of a Transformer:
    - num_params: number of parameters (in bytes)
    - memory: total memory usage (parameters + KV cache) in bytes
    - latency: time to generate one token (seconds/token)
    - throughput: tokens generated per second
    """
    num_params: int
    memory: int
    latency: float
    throughput: float

    def substitute(self, key, value):
        """Substitute `key` with `value` in all stats."""
        return TransformerPerformanceStats(
            self.num_params.subs(key, value).simplify(),
            self.memory.subs(key, value).simplify(),
            self.latency.subs(key, value).simplify(),
            self.throughput.subs(key, value).simplify(),
        )


def compute_transformer_performance_stats(config) -> TransformerPerformanceStats:
    """Compute various performance stats for the Transformer given `config`."""

    # Number of parameters in the Transformer
    num_params = 2*V*D + D*F*3*L + (2*D*N*H + 2*D*K*H)*L

    # How much memory the parameters take
    parameter_size = 2*num_params  # 2 for bf16 (training requires a larger multiple)

    # How much the KV cache takes per sequence (S tokens, K heads, H head dim, L layers)
    kv_cache_size_per_seq = S * (K*H) * L * 2 * 2  # 2 for key + value, 2 for bf16

    # Total memory usage
    memory = B * kv_cache_size_per_seq + parameter_size

    # *Latency* is determined by memory IO (read all parameters and KV cache for each step)
    latency = memory / memory_bandwidth

    # *Throughput* is the inverse of latency, but we're generating B tokens in parallel
```

```
    throughput = B / latency

    # Substitute config
    num_params = num_params.subs(config).simplify()
    memory = memory.subs(config).simplify()
    latency = latency.subs(config).simplify()
    throughput = throughput.subs(config).simplify()

    return TransformerPerformanceStats(num_params, memory, latency, throughput)


def llama2_13b_config(args={}):
    return {
        S: 1024,   # Sequence length
        D: 5120,   # Model dim
        F: 13824,  # Feed-forward dim
        N: 40,     # Number of query heads
        K: 40,     # Number of key/value heads
        H: 128,    # Head dimension
        L: 40,     # Number of layers
        V: 32000,  # Vocabulary size
        memory_bandwidth: 3.35e12,  # Memory bandwidth (bytes/second)
        **args
    }


def throughput_and_latency():
```

So we have shown that inference is memory-bound.
Let us now compute the theoretical maximum latency and throughput of a single request.
Assumption: can overlap compute and communication perfectly and ignore overhead.

Instantiate latency and throughput for Llama 2 13B on an H100:

```
    config = llama2_13b_config()
    stats = compute_transformer_performance_stats(config)

    # Batch size 1
    b1 = stats.substitute(B, 1)

    # Batch size 64
    b64 = stats.substitute(B, 64)
```

Result: worse latency, better throughput

```
    # Batch size 256
    b256 = stats.substitute(B, 256)
```

Result: even worse latency, even better throughput

```
    h100_memory = 80e9  # H100 memory in bytes
    assert b256.memory > h100_memory  # Doesn't fit in memory!
```

Result: doesn't fit into memory and throughput gains are diminishing too...

What increasing batch size does:

- Worsens latency because larger KV cache (O(B) size) to read/write
- Improves throughput because amortizes the cost of reading parameters

Tradeoff between latency and throughput:
1. Smaller batch sizes yield better latency but worse throughput
2. Larger batch sizes yield better throughput but worse latency

Easy parallelism: if you launch M copies of the model, latency is the same, throughput increases by M!

Harder parallelism: shard the model and the KV cache [Scaling book chapter on inference]

Note: time-to-first-token (TTFT) is essentially a function of prefill time
Use smaller batch sizes during prefill for faster TTFT
Use larger batch sizes during generation to improve throughput

```
def reduce_kv_cache_size():
```

Recall that memory is the bottleneck for inference.
So let's try to reduce the size of the KV cache
...but make sure we don't lose too much accuracy.

Grouped-query attention (GQA)

[Ainslie+ 2023]

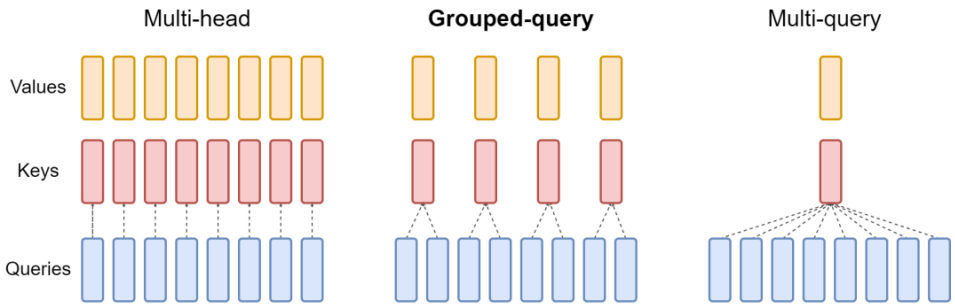
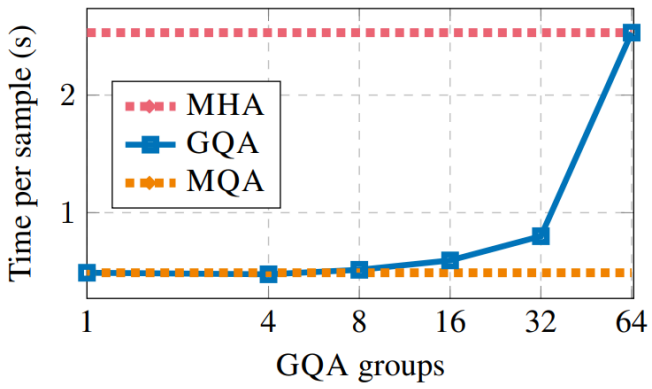


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

Idea: N query heads, but only K key and value heads, each interacting with N/K query heads
Multi-headed attention (MHA): K=N
Multi-query attention (MQA): K=1
Group-query attention (GQA): K is somewhere in between

Latency/throughput improves: [Ainslie+ 2023]



Why does GQA improve latency and throughput?
GQA reduces the KV cache by a factor of N/K.
Reminder: reducing memory usage leads to speedup (since we're memory-bound).

```
# Original Llama 2 13B (MHA)
config = llama2_13b_config({K: 40, B: 64})
k40_b64 = compute_transformer_performance_stats(config)

# GQA with 1:5 ratio (K:N)
```

```
config = llama2_13b_config({K: 8, B: 64})  # Use GQA with 1:5 ratio
k8_b64 = compute_transformer_performance_stats(config)
```

Result: Worse latency, but better throughput (and it fits in memory now!)

```
# Now we can increase the batch size
config = llama2_13b_config({K: 8, B: 256})  # Increase batch size
k8_b256 = compute_transformer_performance_stats(config)
```

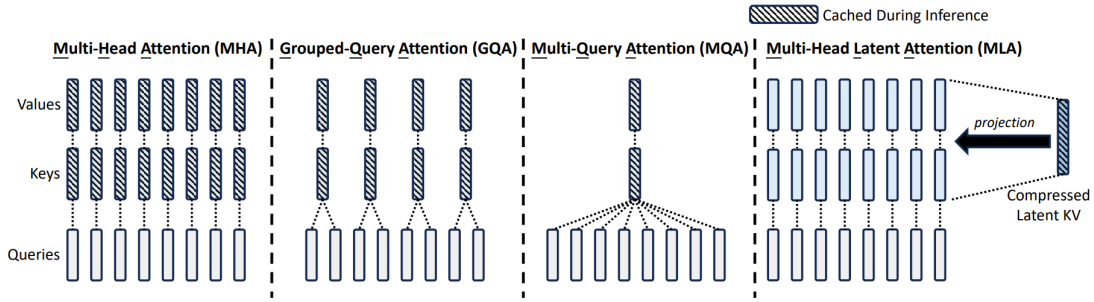
Result: Worse latency, but better throughput (and still fits in memory!)

Check that accuracy doesn't drop: [Ainslie+ 2023]

Model	T_{infer}	Average	CNN	arXiv	PubMed	MediaSum	MultiNews	WMT	TriviaQA
	s		R_1	R_1	R_1	R_1	R_1	BLEU	F1
MHA-Large	0.37	46.0	42.9	44.6	46.2	35.5	46.6	27.7	78.2
MHA-XXL	1.51	47.2	43.8	45.6	47.5	36.4	46.9	28.4	81.9
MQA-XXL	0.24	46.6	43.0	45.0	46.9	36.1	46.5	28.5	81.3
GQA-8-XXL	0.28	47.1	43.5	45.4	47.7	36.3	47.2	28.4	81.6

Multi-head latent attention (MLA)

[DeepSeek-AI+ 2024]



Normal attention: KV cache consists of K = W_K h, V = W_V h (N*H dimensions)
MLA: store compressed vector c = W_c h (C dimensions), project up to K = W_K c, V = W_V c when needed
DeepSeek v2: reduce N*H = 16384 to C = 512
Wrinkle: MLA is not compatible with RoPE, so need to add additional 64 dimensions for RoPE, so 512 + 64 = 576 total dimensions
Latency/throughput improvements follow similarly from the KV cache reduction as argued earlier

Let's now check the accuracy.
First, MHA is better than GQA (though more expensive) [Table 8] [DeepSeek-AI+ 2024]

Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	**37.0**
MMLU (Acc.)	5-shot	37.9	41.2	**45.2**
C-Eval (Acc.)	5-shot	30.0	37.7	**42.9**
CMMLU (Acc.)	5-shot	34.6	38.4	**43.5**

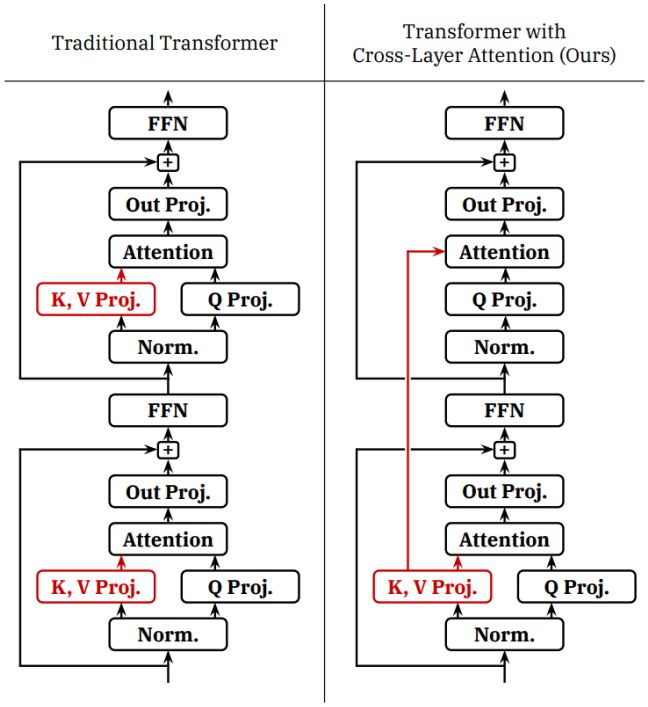
Table 8 | Comparison among 7B dense models with MHA, GQA, and MQA, respectively. MHA demonstrates significant advantages over GQA and MQA on hard benchmarks.

Second, MLA is even a bit better than MHA (and much cheaper) [Table 9] [DeepSeek-AI+ 2024]

Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	**39.0**	46.6	**50.7**
MMLU (Acc.)	5-shot	48.7	**50.0**	57.5	**59.0**
C-Eval (Acc.)	5-shot	**51.6**	50.9	57.9	**59.2**
CMMLU (Acc.)	5-shot	52.3	**53.4**	60.7	**62.5**

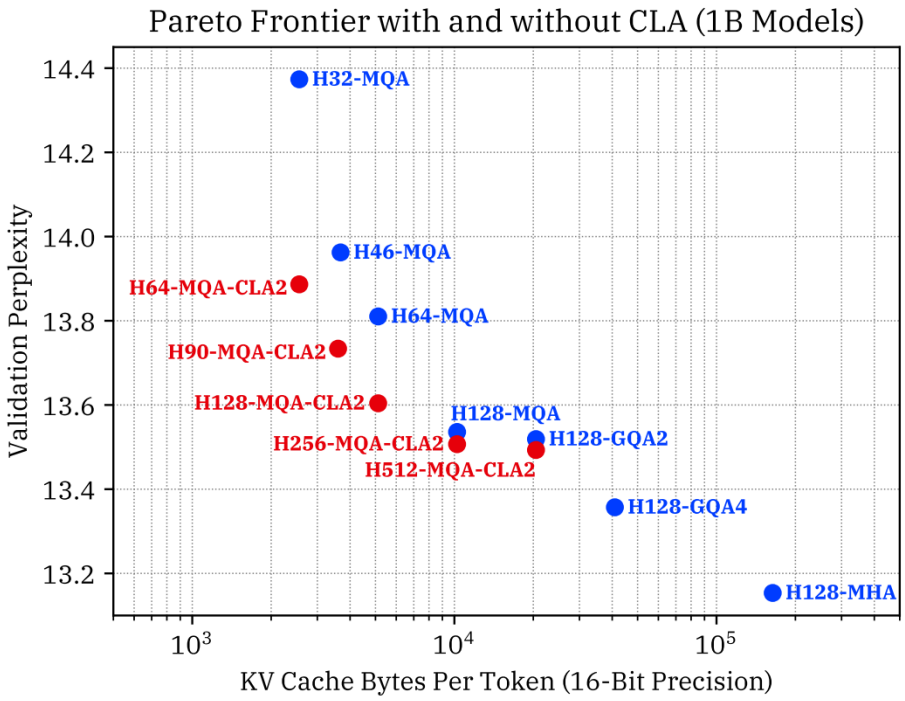
Cross-layer attention (CLA)

[Brandon+ 2024]



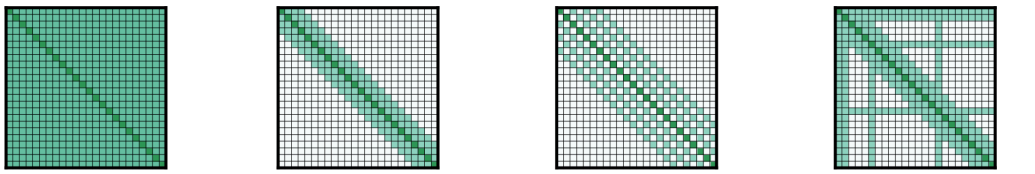
Idea: share KVs across **layers** (just as GQA shares KVs across heads)

Empirically improves the pareto frontier of accuracy and KV cache size (latency and throughput)



Local (sliding window) attention

[Beltagy+ 2020][Child+ 2019][Jiang+ 2023]

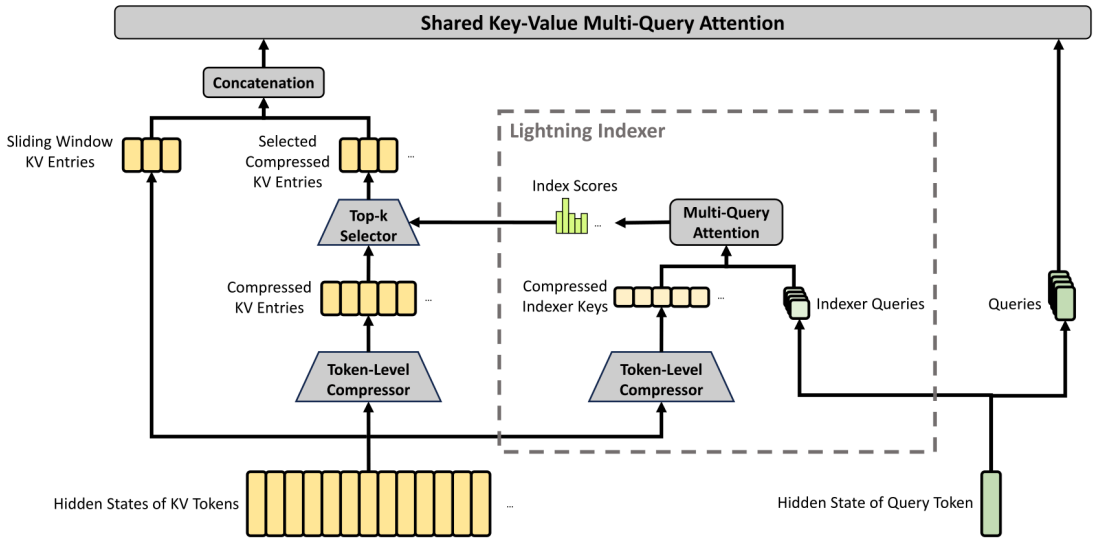


(a) Full n^2 attention (b) Sliding window attention (c) Dilated sliding window (d) Global+sliding window

Idea: just look at the local context, which is most relevant for modeling
Effective context scales linearly with the number of layers
KV cache is independent of sequence length!
Problem: this can still hurt accuracy
Solution: interleave local attention with global attention (hybrid layers)

DeepSeek v4 attention

- Supports 1M context length [DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence]



- Compressed Sparse Attention (CSA): compresses every m tokens into 1
- DeepSeek Sparse Attention (DSA): selects the top k
- Heavily Compressed Attention (HCA): compresses even more

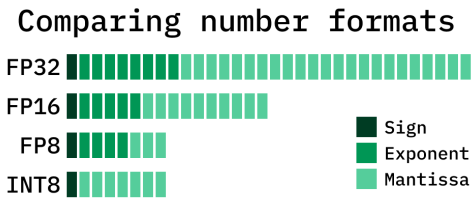
Summary:

- Goal: reduce the KV cache size (since inference is memory-bound) without hurting accuracy
- Lower-dimensional KV cache (GQA, MLA, CLA)
- Local attention (truncates the KV cache) on some of the layers
- Other ideas: linear attention / state-space-models (Mamba 2, GatedDeltaNet), diffusion models

```
def quantization():
```

Key idea: reduce the precision of numbers
Less memory means higher latency/throughput (since inference is memory-bound).
Of course we have to worry about accuracy...

```
# Mechanics
x = 5.2342
scale = 0.1
zero_point = 4
x_quant = round(x / scale) + zero_point  # Quantize
x_approx = (x_quant - zero_point) * scale  # Dequantize
```



[article]

- fp32 (4 bytes): needed for parameters and optimizer states during training
- bf16 (2 bytes): default for inference
- fp8 (1 byte) [-240, 240] for e4m3 on H100s: can train if you dare [Peng+ 2023]
- int8 (1 byte) [-128, 127]: less accurate but cheaper than fp8, but for inference only [Baalen+ 2023]
- int4 (0.5 bytes) [-8, 7]: cheaper, even less accurate [Baalen+ 2023]

[Overview of approaches]

Quantization-aware training (QAT)

- During training, quantize-and-dequantize during the forward pass to simulate quantization errors
- Pro: weights are trained to work with quantization

- Con: requires expensive large-scale training

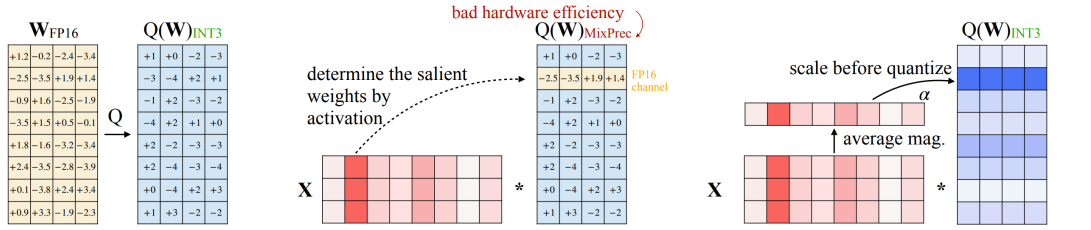
Post-training quantization (PTQ):

- Done after training, so much cheaper
- Run on sample data to determine scale and zero point for each layer or tensor
- GPTQ: use Hessian information to update non-quantized weights to account for quantization error [Frantar+ 2022]

Activation-aware quantization (AWQ)

[Lin+ 2023]

- Observation: some activation channels are large
- Weights that hit those matter more
- Allocate more precision to those weights
- Idea: select which weights (0.1-1%) to keep in high precision based on activations
- fp16 → int3 produces 4x lower memory, 3.2x speedup



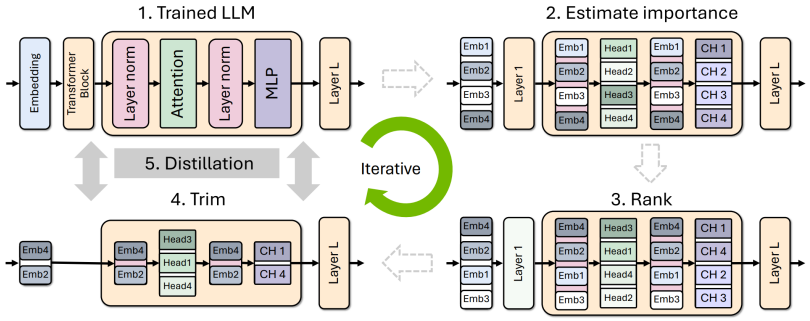
(a) RTN quantization (**PPL 43.2**) (b) Keep 1% salient weights in FP16 (**PPL 13.0**) (c) Scale the weights before quantization (**PPL 13.0**)

```
def model_pruning():
```

Key idea: just rip out parts of an expensive model to make it cheaper

...and then fix it up.

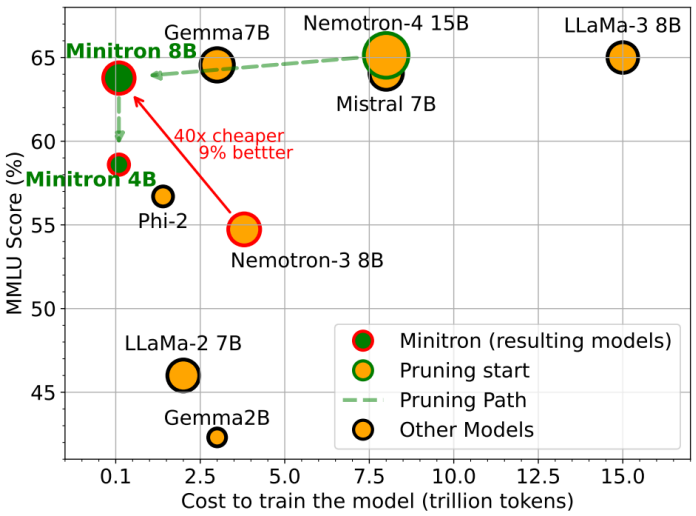
Paper from NVIDIA [Muralidharan+ 2024]



Algorithm:

1. Identify important {layer, head, hidden dimension} on a small calibration dataset (1024 samples)
2. Remove unimportant layers to get a smaller model
3. Distill the original model into pruned model

Results:




```
# TODO
```

```
def speculative_sampling():
```

Recall the two stages of inference:

- Prefill: given a sequence, encode tokens in parallel (compute-bound) [note: also gives you probabilities]
- Generation: generate one token at a time (memory-bound)

In other words, checking is faster than generation.

Speculative sampling [Leviathan+ 2022][Chen+ 2023]

- Use a cheaper **draft model** p to guess a few tokens (e.g., 4)
- Evaluate with target model q (process tokens in parallel), and accept if it looks good

[Speculative sampling video]
[article]

Algorithm 2 Speculative Sampling (SpS) with Auto-Regressive Target and Draft Models

Given lookahead K and minimum target sequence length T.
Given auto-regressive target model $q(.|.)$, and auto-regressive draft model $p(.|.)$, initial prompt sequence $x_0, \ldots, x_t$.
Initialise $n \leftarrow t$.
while $n < T$ **do**
  for $t = 1 : K$ **do**
    Sample draft auto-regressively $\tilde{x}_t \sim p(x|, x_1, \ldots, x_n, \tilde{x}_1, \ldots, \tilde{x}_{t-1})$
  end for
  In parallel, compute $K + 1$ sets of logits from drafts $\tilde{x}_1, \ldots, \tilde{x}_K$:

$$q(x|, x_1, \ldots, x_n),\ q(x|, x_1, \ldots, x_n, \tilde{x}_1),\ \ldots,\ q(x|, x_1, \ldots, x_n, \tilde{x}_1, \ldots, \tilde{x}_K)$$

  for $t = 1 : K$ **do**
    Sample $r \sim U[0, 1]$ from a uniform distribution.
    if $r < \min\left(1, \frac{q(x|x_1,\ldots,x_{n+t-1})}{p(x|x_1,\ldots,x_{n+t-1})}\right)$, **then**
      Set $x_{n+t} \leftarrow \tilde{x}_t$ and $n \leftarrow n + 1$.
    else
      sample $x_{n+t} \sim (q(x|x_1, \ldots, x_{n+t-1}) - p(x|x_1, \ldots, x_{n+t-1}))_+$ and exit for loop.
    end if
  end for
  If all tokens $x_{n+1}, \ldots, x_{n+K}$ are accepted, sample extra token $x_{n+K+1} \sim q(x|, x_1, \ldots, x_n, x_{n+K})$ and set $n \leftarrow n + 1$.
end while

This is modified rejection sampling with proposal p and target q
Modification: always generate at least one candidate (rejection sampling will keep looping)
Key property: guaranteed to be an **exact sample** from the target model!

Proof by example: assume two vocabulary elements {A, B}

- Target model probabilities: [q(A), q(B)]
- Draft model probabilities: [p(A), p(B)]

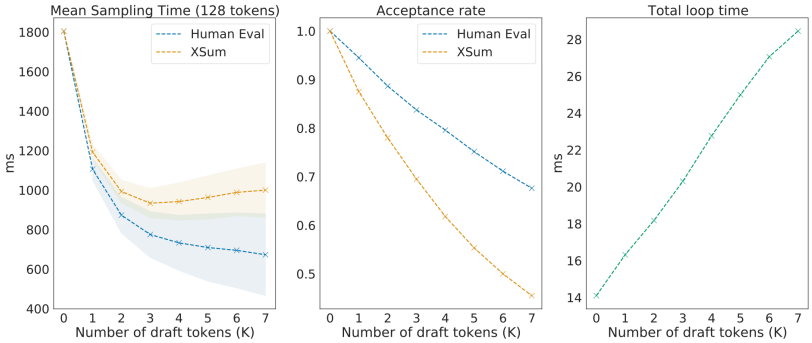
- Assume p(A) > q(A) [draft model oversamples A].
- Therefore p(B) < q(B) [draft model undersamples B].
- Residual probabilities max(q-p, 0): [0, 1]

Compute the probabilities of speculatively sampling a token:

- P[sampling A] = p(A) * (q(A) / p(A)) + p(B) * 1 * 0 = q(A)
- P[sampling B] = p(B) * 1 + p(A) * (1 - q(A) / p(A)) * 1 = q(B)

Table 1 | **Chinchilla performance and speed on XSum and HumanEval with naive and speculative sampling at batch size 1 and** $K = 4$. XSum was executed with nucleus parameter $p = 0.8$, and HumanEval with $p = 0.95$ and temperature 0.8.

Sampling Method	Benchmark	Result	Mean Token Time	Speed Up
ArS (Nucleus)	XSum (ROUGE-2)	0.112	14.1ms/Token	1×
SpS (Nucleus)		0.114	7.52ms/Token	1.92×
ArS (Greedy)	XSum (ROUGE-2)	0.157	14.1ms/Token	1×
SpS (Greedy)		0.156	7.00ms/Token	2.01×
ArS (Nucleus)	HumanEval (100 Shot)	45.1%	14.1ms/Token	1×
SpS (Nucleus)		47.0%	5.73ms/Token	2.46×

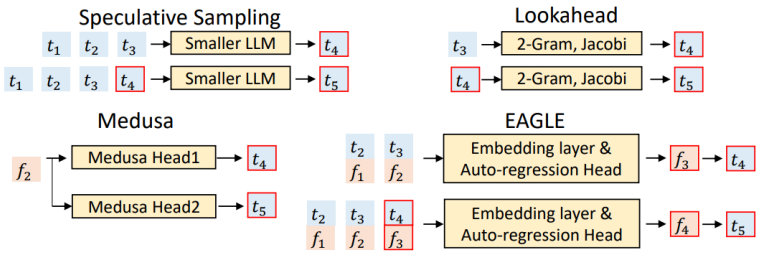


In practice:

- Target model has 70B parameters, draft model has 8B parameters
- Target model has 8B parameters, draft model has 1B parameters
- Try to make draft model as close to target (distillation)

Extensions to improve the draft model:

- Medusa: draft model generates multiple tokens in parallel [Cai+ 2024]
- EAGLE: draft model takes high-level features from target model [Li+ 2024]



Summary:

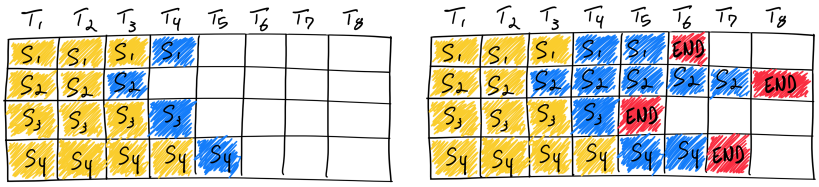
- Exact sampling from target model (thanks to math)!
- Exploits asymmetry between checking and generation
- Lots of room for innovation on the draft model (involves training)

```
def continuous_batching():
```

[Orca: A Distributed Serving System for Transformer-Based Generative Models][talk]

Problem:

- Training: get a dense block of tokens (batch size x sequence length)
- Inference: requests arrive and finish at different times, so you have a ragged array



Solution: iteration-level scheduling

- Decode step by step
- Add new requests to the batch as they arrive (so don't have to wait until generation completes)

Problem:

- Batching only works when all sequences have the same dimensionality (right?)
- But each request might have a different length

Solution: selective batching

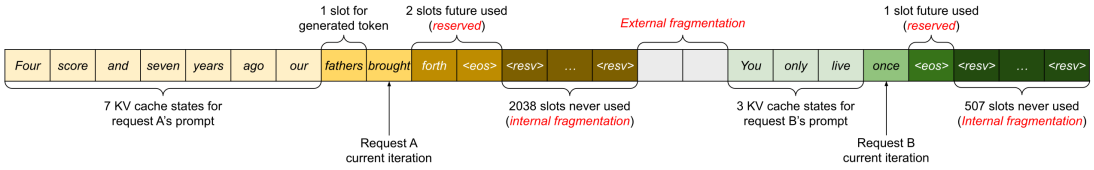
- Training: when all sequences of the same length, operate on a B x S x H tensor
- But we might have different lengths: [3, H], [9, H], [5, H], etc.
- Attention computation: process each sequence separately
- Non-attention computation: concatenate all the sequences together to [3 + 9 + 5, H]

`def paged_attention():`

Paper that introduced vLLM in addition to PagedAttention [Kwon+ 2023]

Previous status quo:

- Request comes in
- Allocate section of KV cache for prompt and response (up to a max length)

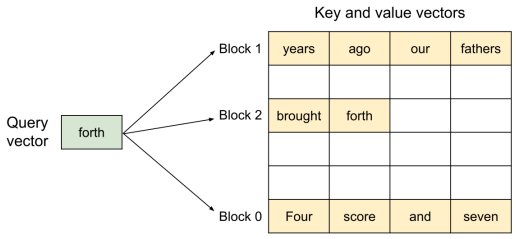


Problem: fragmentation (what happens to your hard drive)

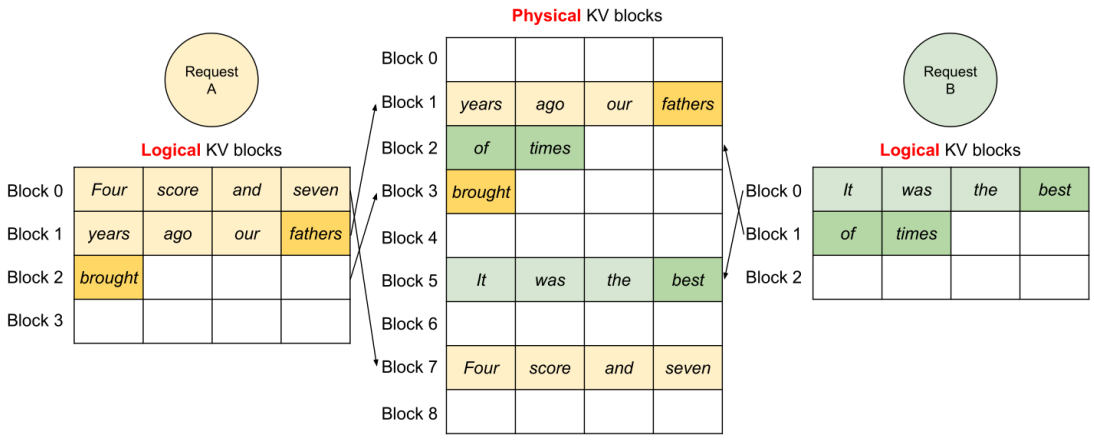
- But this is wasteful since we might generate much fewer tokens (internal fragmentation)!
- Might be extra unused space between sections (external fragmentation)!

Solution: PagedAttention (remember operating systems)

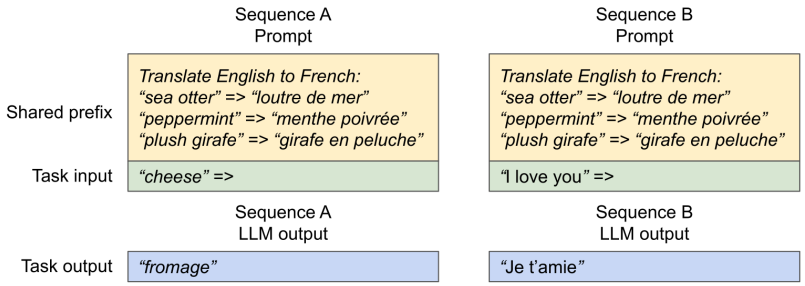
- Divide the KV cache of a sequence into non-contiguous **blocks**



Two requests share the KV caches:

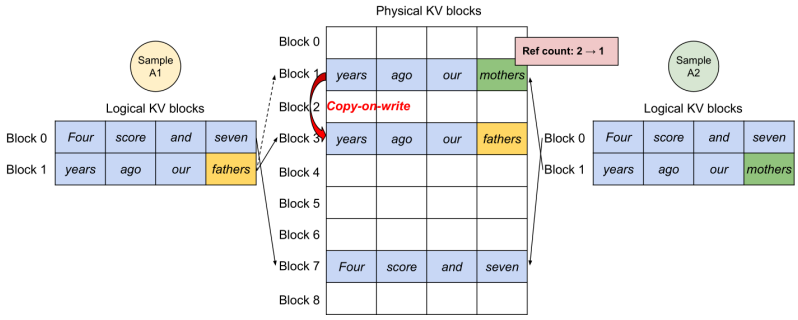


In general, multiple types of sharing KV caches across sequences:



- Sharing the system prompt
- Sampling multiple responses per prompt (e.g., for program synthesis)

Solution: share prefixes, copy-on-write at the block level



Other vLLM optimizations:

- Kernel to fuse block read and attention (reduce kernel launch overhead)
- Use latest kernels (FlashAttention, FlashDecoding)
- Use CUDA graphs to avoid kernel launch overhead

Summary: use ideas from operating systems (paging) to make use of memory for dynamic workloads

```
if __name__ == "__main__":
    main()
```